

# 405 report coding for model and HMC

Lijie Wang-37325834

2026-04-07

```
library(dplyr)
```

```
##  
## Attaching package: 'dplyr'  
  
## The following objects are masked from 'package:stats':  
##  
##   filter, lag  
  
## The following objects are masked from 'package:base':  
##  
##   intersect, setdiff, setequal, union
```

```
library(readr)  
library(posterior)
```

```
## This is posterior version 1.6.1.9000  
  
##  
## Attaching package: 'posterior'  
  
## The following objects are masked from 'package:stats':  
##  
##   mad, sd, var  
  
## The following objects are masked from 'package:base':  
##  
##   %in%, match
```

```
library(loo)
```

```
## Warning: package 'loo' was built under R version 4.3.3  
  
## This is loo version 2.8.0  
  
## - Online documentation and vignettes at mc-stan.org/loo  
  
## - As of v2.0.0 loo defaults to 1 core but we recommend using as many as possible. Use the 'cores' arg
```

```
##
## Attaching package: 'loo'
```

```
## The following object is masked from 'package:posterior':
##
##      gpdfit
```

```
library(cmdstanr)
```

```
## This is cmdstanr version 0.9.0

## - CmdStanR documentation and vignettes: mc-stan.org/cmdstanr

## - CmdStan path: /Users/cindy-wang/.cmdstan/cmdstan-2.38.0

## - CmdStan version: 2.38.0
```

```
df <- read_csv("data.csv")
```

```
## Rows: 2577 Columns: 11
## -- Column specification -----
## Delimiter: ","
## chr (1): playerName
## dbl (10): birthYear, debutYear, finalYear, yearID, age, age2, AB, H, batting...
##
## i Use 'spec()' to retrieve the full column specification for this data.
## i Specify the column types or set 'show_col_types = FALSE' to quiet this message.
```

```
glimpse(df)
```

```
## Rows: 2,577
## Columns: 11
## $ playerName <chr> "Bobby Abreu", "Bobby Abreu", "Bobby Abreu", "Bobby Abreu"~
## $ birthYear <dbl> 1974, 1974, 1974, 1974, 1974, 1974, 1974, 1974, 1974, 1974~
## $ debutYear <dbl> 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996, 1996~
## $ finalYear <dbl> 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014, 2014~
## $ yearID <dbl> 1997, 1998, 1999, 2000, 2001, 2002, 2003, 2004, 2005, 2006~
## $ age <dbl> 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37~
## $ age2 <dbl> 529, 576, 625, 676, 729, 784, 841, 900, 961, 1024, 1089, 1~
## $ AB <dbl> 188, 497, 546, 576, 588, 572, 577, 574, 588, 548, 605, 609~
## $ H <dbl> 47, 155, 183, 182, 170, 176, 173, 173, 168, 163, 171, 180,~
## $ batting_avg <dbl> 0.2500000, 0.3118712, 0.3351648, 0.3159722, 0.2891156, 0.3~
## $ n_seasons <dbl> 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17, 17~
```

```
head(df)
```

```
## # A tibble: 6 x 11
##   playerName birthYear debutYear finalYear yearID age age2 AB H
##   <chr>         <dbl>    <dbl>    <dbl>   <dbl> <dbl> <dbl> <dbl> <dbl>
## 1 Bobby Abreu    1974      1996      2014    1997    23   529   188   47
```

```
## 2 Bobby Abreu      1974      1996      2014      1998      24      576      497      155
## 3 Bobby Abreu      1974      1996      2014      1999      25      625      546      183
## 4 Bobby Abreu      1974      1996      2014      2000      26      676      576      182
## 5 Bobby Abreu      1974      1996      2014      2001      27      729      588      170
## 6 Bobby Abreu      1974      1996      2014      2002      28      784      572      176
## # i 2 more variables: batting_avg <dbl>, n_seasons <dbl>
```

```
summary(df[, c("age", "AB", "H", "batting_avg")])
```

```
##           age           AB           H           batting_avg
##  Min.      :19.00   Min.      :100.0   Min.      : 15.0   Min.      :0.1418
## 1st Qu.:27.00   1st Qu.:282.0   1st Qu.: 72.0   1st Qu.:0.2466
## Median :30.00   Median :451.0   Median :119.0   Median :0.2702
## Mean   :30.09   Mean   :418.5   Mean   :114.7   Mean   :0.2683
## 3rd Qu.:33.00   3rd Qu.:557.0   3rd Qu.:156.0   3rd Qu.:0.2913
## Max.   :44.00   Max.   :716.0   Max.   :262.0   Max.   :0.3945
```

```
any(df$H > df$AB)
```

```
## [1] FALSE
```

```
df <- df %>%
  mutate(
    player_id = as.integer(factor(playerName)),
    age_mean = mean(age),
    age_c = age - age_mean,
    age_c2 = age_c^2
  ) %>%
  arrange(player_id, yearID)
```

```
head(df[, c("playerName", "player_id", "yearID", "age", "age_c", "age_c2", "AB", "H")])
```

```
## # A tibble: 6 x 8
##   playerName      player_id yearID   age   age_c   age_c2   AB   H
##   <chr>          <int>   <dbl> <dbl>   <dbl>   <dbl> <dbl> <dbl>
## 1 A. J. Pierzynski      1    2001    25 -5.09    25.9    381   110
## 2 A. J. Pierzynski      1    2002    26 -4.09    16.7    440   132
## 3 A. J. Pierzynski      1    2003    27 -3.09     9.54    487   152
## 4 A. J. Pierzynski      1    2004    28 -2.09     4.36    471   128
## 5 A. J. Pierzynski      1    2005    29 -1.09     1.19    460   118
## 6 A. J. Pierzynski      1    2006    30 -0.0893  0.00797  509   150
```

```
summary(df[, c("age", "age_c", "age_c2")])
```

```
##           age           age_c           age_c2
##  Min.      :19.00   Min.      :-11.08925   Min.      : 0.00797
## 1st Qu.:27.00   1st Qu.: -3.08925   1st Qu.:  3.65096
## Median :30.00   Median : -0.08925   Median :  9.54347
## Mean   :30.09   Mean   :  0.00000   Mean   : 19.35486
## 3rd Qu.:33.00   3rd Qu.:  2.91075   3rd Qu.: 25.90048
## Max.   :44.00   Max.   : 13.91075   Max.   :193.50894
```

```
length(unique(df$player_id))
```

```
## [1] 200
```

```
N <- nrow(df)
J <- length(unique(df$player_id))
age_mean <- mean(df$age)
```

```
#Model 1
```

```
stan_data_m1 <- list(
  N = N,
  H = df$H,
  AB = df$AB,
  age = df$age_c,
  age2 = df$age_c2
)
```

```
mod1 <- cmdstan_model("model1.stan")
```

```
fit1 <- mod1$sample(
  data = stan_data_m1,
  seed = 1,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000
)
```

```
## Running MCMC with 4 parallel chains...
```

```
##
```

```
## Chain 1 Iteration:    1 / 2000 [ 0%] (Warmup)
```

```
## Chain 2 Rejecting initial value:
```

```
## Chain 2   Log probability evaluates to log(0), i.e. negative infinity.
```

```
## Chain 2   Stan can't start sampling from this initial value.
```

```
## Chain 2 Iteration:    1 / 2000 [ 0%] (Warmup)
```

```
## Chain 3 Iteration:    1 / 2000 [ 0%] (Warmup)
```

```
## Chain 3 Iteration:  100 / 2000 [ 5%] (Warmup)
```

```
## Chain 4 Rejecting initial value:
```

```
## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.
```

```
## Chain 4   Stan can't start sampling from this initial value.
```

```
## Chain 4 Rejecting initial value:
```

```

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

## Chain 4 Rejecting initial value:

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

## Chain 4 Iteration:    1 / 2000 [  0%] (Warmup)
## Chain 4 Iteration:   100 / 2000 [  5%] (Warmup)
## Chain 1 Iteration:   100 / 2000 [  5%] (Warmup)
## Chain 2 Iteration:   100 / 2000 [  5%] (Warmup)
## Chain 3 Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 4 Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 2 Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 3 Iteration:   300 / 2000 [ 15%] (Warmup)
## Chain 1 Iteration:   200 / 2000 [ 10%] (Warmup)
## Chain 4 Iteration:   300 / 2000 [ 15%] (Warmup)
## Chain 3 Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 1 Iteration:   300 / 2000 [ 15%] (Warmup)
## Chain 2 Iteration:   300 / 2000 [ 15%] (Warmup)
## Chain 4 Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 3 Iteration:   500 / 2000 [ 25%] (Warmup)
## Chain 1 Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 2 Iteration:   400 / 2000 [ 20%] (Warmup)
## Chain 4 Iteration:   500 / 2000 [ 25%] (Warmup)
## Chain 3 Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 4 Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 1 Iteration:   500 / 2000 [ 25%] (Warmup)
## Chain 2 Iteration:   500 / 2000 [ 25%] (Warmup)
## Chain 3 Iteration:   700 / 2000 [ 35%] (Warmup)
## Chain 4 Iteration:   700 / 2000 [ 35%] (Warmup)
## Chain 1 Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 2 Iteration:   600 / 2000 [ 30%] (Warmup)
## Chain 3 Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 4 Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 1 Iteration:   700 / 2000 [ 35%] (Warmup)
## Chain 2 Iteration:   700 / 2000 [ 35%] (Warmup)
## Chain 3 Iteration:   900 / 2000 [ 45%] (Warmup)
## Chain 4 Iteration:   900 / 2000 [ 45%] (Warmup)
## Chain 1 Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 2 Iteration:   800 / 2000 [ 40%] (Warmup)
## Chain 3 Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 3 Iteration:  1001 / 2000 [ 50%] (Sampling)
## Chain 4 Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 4 Iteration:  1001 / 2000 [ 50%] (Sampling)
## Chain 1 Iteration:   900 / 2000 [ 45%] (Warmup)
## Chain 2 Iteration:   900 / 2000 [ 45%] (Warmup)
## Chain 3 Iteration:  1100 / 2000 [ 55%] (Sampling)
## Chain 2 Iteration:  1000 / 2000 [ 50%] (Warmup)
## Chain 1 Iteration:  1000 / 2000 [ 50%] (Warmup)

```

```

## Chain 1 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 4 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 1 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 2 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 4 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 3 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 3 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 2 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3 finished in 11.7 seconds.
## Chain 2 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4 finished in 12.2 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 2 finished in 12.9 seconds.
## Chain 1 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 1 finished in 13.0 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 12.5 seconds.
## Total execution time: 13.1 seconds.

```

```
fit1$summary(c("beta0", "beta1", "beta2"))
```

```

## # A tibble: 3 x 10
##   variable      mean    median      sd      mad      q5      q95  rhat ess_bulk

```

```
##   <chr>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl>      <dbl> <dbl>      <dbl>
## 1 beta0    -0.960      -0.960      0.00274  2.77e-3 -0.964    -9.55e-1  1.01      655.
## 2 beta1    -0.00614    -0.00614    0.000526 5.30e-4 -0.00702 -5.27e-3  1.00      2323.
## 3 beta2    -0.000855   -0.000854   0.000103 1.05e-4 -0.00103 -6.89e-4  1.00      1385.
## # i 1 more variable: ess_tail <dbl>
```

```
draws1 <- fit1$draws(c("beta0", "beta1", "beta2"), format = "df")
```

*#Model 2*

```
stan_data_m2 <- list(
  N = N,
  J = J,
  H = df$H,
  AB = df$AB,
  player = df$player_id,
  age = df$age_c,
  age2 = df$age_c2
)
```

```
mod2 <- cmdstan_model("model2.stan")
```

```
fit2 <- mod2$sample(
  data = stan_data_m2,
  seed = 1,
  chains = 4,
  parallel_chains = 4,
  iter_warmup = 1000,
  iter_sampling = 1000
)
```

```
## Running MCMC with 4 parallel chains...
```

```
## Chain 1 Rejecting initial value:
```

```
## Chain 1   Log probability evaluates to log(0), i.e. negative infinity.
```

```
## Chain 1   Stan can't start sampling from this initial value.
```

```
## Chain 1 Rejecting initial value:
```

```
## Chain 1   Log probability evaluates to log(0), i.e. negative infinity.
```

```
## Chain 1   Stan can't start sampling from this initial value.
```

```
## Chain 1 Rejecting initial value:
```

```
## Chain 1   Log probability evaluates to log(0), i.e. negative infinity.
```

```
## Chain 1   Stan can't start sampling from this initial value.
```

```

## Chain 1 Iteration:      1 / 2000 [  0%]   (Warmup)

## Chain 1 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 1 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/h9/1x8')
## Chain 1 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 1 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 1

## Chain 2 Rejecting initial value:

## Chain 2   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 2   Stan can't start sampling from this initial value.

## Chain 2 Iteration:      1 / 2000 [  0%]   (Warmup)
## Chain 3 Iteration:      1 / 2000 [  0%]   (Warmup)

## Chain 3 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 3 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/h9/1x8')
## Chain 3 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 3 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.
## Chain 3

## Chain 4 Rejecting initial value:

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

## Chain 4 Rejecting initial value:

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

## Chain 4 Rejecting initial value:

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

```



```

## Chain 4 Rejecting initial value:

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

## Chain 4 Rejecting initial value:

## Chain 4   Log probability evaluates to log(0), i.e. negative infinity.

## Chain 4   Stan can't start sampling from this initial value.

## Chain 4 Iteration:      1 / 2000 [  0%]   (Warmup)

## Chain 4 Informational Message: The current Metropolis proposal is about to be rejected because of the
## Chain 4 Exception: normal_lpdf: Scale parameter is 0, but must be positive! (in '/var/folders/h9/1x8')
## Chain 4 If this warning occurs sporadically, such as for highly constrained variable types like covariance
## Chain 4 but if this warning occurs often then your model may be either severely ill-conditioned or misspecified.

## Chain 4

## Chain 4 Iteration:  100 / 2000 [  5%]   (Warmup)
## Chain 1 Iteration:  100 / 2000 [  5%]   (Warmup)
## Chain 2 Iteration:  100 / 2000 [  5%]   (Warmup)
## Chain 4 Iteration:  200 / 2000 [ 10%]   (Warmup)
## Chain 1 Iteration:  200 / 2000 [ 10%]   (Warmup)
## Chain 3 Iteration:  100 / 2000 [  5%]   (Warmup)
## Chain 2 Iteration:  200 / 2000 [ 10%]   (Warmup)
## Chain 4 Iteration:  300 / 2000 [ 15%]   (Warmup)
## Chain 1 Iteration:  300 / 2000 [ 15%]   (Warmup)
## Chain 2 Iteration:  300 / 2000 [ 15%]   (Warmup)
## Chain 3 Iteration:  200 / 2000 [ 10%]   (Warmup)
## Chain 4 Iteration:  400 / 2000 [ 20%]   (Warmup)
## Chain 1 Iteration:  400 / 2000 [ 20%]   (Warmup)
## Chain 2 Iteration:  400 / 2000 [ 20%]   (Warmup)
## Chain 4 Iteration:  500 / 2000 [ 25%]   (Warmup)
## Chain 1 Iteration:  500 / 2000 [ 25%]   (Warmup)
## Chain 3 Iteration:  300 / 2000 [ 15%]   (Warmup)
## Chain 2 Iteration:  500 / 2000 [ 25%]   (Warmup)
## Chain 4 Iteration:  600 / 2000 [ 30%]   (Warmup)
## Chain 1 Iteration:  600 / 2000 [ 30%]   (Warmup)
## Chain 2 Iteration:  600 / 2000 [ 30%]   (Warmup)
## Chain 3 Iteration:  400 / 2000 [ 20%]   (Warmup)
## Chain 4 Iteration:  700 / 2000 [ 35%]   (Warmup)
## Chain 1 Iteration:  700 / 2000 [ 35%]   (Warmup)
## Chain 2 Iteration:  700 / 2000 [ 35%]   (Warmup)
## Chain 4 Iteration:  800 / 2000 [ 40%]   (Warmup)
## Chain 1 Iteration:  800 / 2000 [ 40%]   (Warmup)

```

```

## Chain 3 Iteration: 500 / 2000 [ 25%] (Warmup)
## Chain 2 Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 4 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 1 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 2 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 3 Iteration: 600 / 2000 [ 30%] (Warmup)
## Chain 4 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 4 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 1 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 1 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 2 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 3 Iteration: 700 / 2000 [ 35%] (Warmup)
## Chain 4 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 1 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 2 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 4 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 1 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 3 Iteration: 800 / 2000 [ 40%] (Warmup)
## Chain 2 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 4 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 1 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 2 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 3 Iteration: 900 / 2000 [ 45%] (Warmup)
## Chain 4 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 1 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 2 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 4 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 1 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 3 Iteration: 1000 / 2000 [ 50%] (Warmup)
## Chain 3 Iteration: 1001 / 2000 [ 50%] (Sampling)
## Chain 2 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 4 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 1 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3 Iteration: 1100 / 2000 [ 55%] (Sampling)
## Chain 2 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 4 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 1 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 3 Iteration: 1200 / 2000 [ 60%] (Sampling)
## Chain 2 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 4 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 1 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3 Iteration: 1300 / 2000 [ 65%] (Sampling)
## Chain 2 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 4 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 3 Iteration: 1400 / 2000 [ 70%] (Sampling)
## Chain 1 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 2 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 4 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 4 finished in 71.9 seconds.
## Chain 3 Iteration: 1500 / 2000 [ 75%] (Sampling)
## Chain 1 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 1 finished in 72.6 seconds.
## Chain 2 Iteration: 2000 / 2000 [100%] (Sampling)

```

```
## Chain 2 finished in 74.0 seconds.
## Chain 3 Iteration: 1600 / 2000 [ 80%] (Sampling)
## Chain 3 Iteration: 1700 / 2000 [ 85%] (Sampling)
## Chain 3 Iteration: 1800 / 2000 [ 90%] (Sampling)
## Chain 3 Iteration: 1900 / 2000 [ 95%] (Sampling)
## Chain 3 Iteration: 2000 / 2000 [100%] (Sampling)
## Chain 3 finished in 84.9 seconds.
##
## All 4 chains finished successfully.
## Mean chain execution time: 75.9 seconds.
## Total execution time: 85.0 seconds.
```

```
fit2$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2"))
```

```
## # A tibble: 4 x 10
##   variable      mean  median      sd      mad      q5      q95  rhat ess_bulk
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl> <dbl>   <dbl>
## 1 mu_alpha  -0.972  -0.972  7.55e-3  7.62e-3 -0.984  -0.959  1.00  4147.
## 2 sigma_alpha 0.0974  0.0971  5.39e-3  5.38e-3  0.0889  0.107  1.00  3807.
## 3 beta1      -0.00851 -0.00852 5.76e-4  5.71e-4 -0.00946 -0.00756 1.00  4213.
## 4 beta2      -0.00146 -0.00146 1.12e-4  1.12e-4 -0.00164 -0.00128 1.00  2499.
## # i 1 more variable: ess_tail <dbl>
```

```
draws2 <- fit2$draws(c("mu_alpha", "sigma_alpha", "beta1", "beta2"), format = "df")
```

```
#Prob of the age effect
mean(draws1$beta1 > 0)
```

```
## [1] 0
```

```
mean(draws1$beta2 < 0)
```

```
## [1] 1
```

```
mean(draws2$beta1 > 0)
```

```
## [1] 0
```

```
mean(draws2$beta2 < 0)
```

```
## [1] 1
```

```
draws1$peak_age <- age_mean + (-draws1$beta1 / (2 * draws1$beta2))
draws2$peak_age <- age_mean + (-draws2$beta1 / (2 * draws2$beta2))
mean(draws1$peak_age)
```

```
## [1] 26.44352
```

```
quantile(draws1$peak_age, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%     97.5%  
## 25.22593 26.48897 27.39060
```

```
mean(draws2$peak_age)
```

```
## [1] 27.15166
```

```
quantile(draws2$peak_age, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%     97.5%  
## 26.48397 27.16868 27.70899
```

```
# whether hierarchical model is justified  
mean(draws2$sigma_alpha)
```

```
## [1] 0.0973507
```

```
quantile(draws2$sigma_alpha, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%     97.5%  
## 0.08744684 0.09714125 0.10844071
```

```
# Comparison  
log_lik1 <- fit1$draws("log_lik", format = "matrix")  
log_lik2 <- fit2$draws("log_lik", format = "matrix")  
  
loo1 <- loo(log_lik1)  
loo2 <- loo(log_lik2)  
  
loo_compare(loo1, loo2)
```

```
##      elpd_diff se_diff  
## model2      0.0      0.0  
## model1 -845.6     56.2
```

```
# Diagnostics  
fit1$diagnostic_summary()
```

```
## $num_divergent  
## [1] 0 0 0 0  
##  
## $num_max_treedepth  
## [1] 0 0 0 0  
##  
## $ebfmi  
## [1] 1.043135 1.061503 1.200593 1.128108
```

```
fit2$diagnostic_summary()
```

```
## $num_divergent
## [1] 0 0 0 0
##
## $num_max_treedepth
## [1] 0 0 0 0
##
## $ebfmi
## [1] 0.9877371 0.9846548 1.0035337 1.0052989
```

Computation by VI-fullrank

```
# Model 1
fit1_vi <- mod1$variational(
  data = stan_data_m1,
  seed = 1,
  algorithm = "fullrank",
  iter = 20000,
  output_samples = 4000
)
```

```
## -----
## EXPERIMENTAL ALGORITHM:
##   This procedure has not been thoroughly tested and may be unstable
##   or buggy. The interface is subject to change.
## -----
## Gradient evaluation took 0.000257 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 2.57 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration: 1 / 250 [ 0%] (Adaptation)
## Iteration: 50 / 250 [ 20%] (Adaptation)
## Iteration: 100 / 250 [ 40%] (Adaptation)
## Iteration: 150 / 250 [ 60%] (Adaptation)
## Iteration: 200 / 250 [ 80%] (Adaptation)
## Iteration: 250 / 250 [100%] (Adaptation)
## Success! Found best value [eta = 0.1].
## Begin stochastic gradient ascent.
```

| iter | ELBO        | delta_ELBO_mean | delta_ELBO_med | notes                            |
|------|-------------|-----------------|----------------|----------------------------------|
| 100  | -117024.603 | 1.000           | 1.000          |                                  |
| 200  | -43919.130  | 1.332           | 1.665          |                                  |
| 300  | -28952.242  | 1.060           | 1.000          |                                  |
| 400  | -12482.859  | 1.125           | 1.319          |                                  |
| 500  | -22433.527  | 0.989           | 1.000          |                                  |
| 600  | -14369.415  | 0.918           | 1.000          |                                  |
| 700  | -11338.207  | 0.825           | 0.561          |                                  |
| 800  | -13626.886  | 0.743           | 0.561          |                                  |
| 900  | -14217.281  | 0.665           | 0.517          |                                  |
| 1000 | -12337.118  | 0.613           | 0.517          |                                  |
| 1100 | -15610.125  | 0.577           | 0.444          | MAY BE DIVERGING... INSPECT ELBO |
| 1200 | -11310.153  | 0.560           | 0.444          | MAY BE DIVERGING... INSPECT ELBO |

|    |      |            |       |       |                                  |
|----|------|------------|-------|-------|----------------------------------|
| ## | 1300 | -12773.990 | 0.526 | 0.380 | MAY BE DIVERGING... INSPECT ELBO |
| ## | 1400 | -12040.229 | 0.493 | 0.380 |                                  |
| ## | 1500 | -16284.081 | 0.477 | 0.267 |                                  |
| ## | 1600 | -14414.902 | 0.456 | 0.267 |                                  |
| ## | 1700 | -10881.777 | 0.448 | 0.267 |                                  |
| ## | 1800 | -11996.321 | 0.428 | 0.267 |                                  |
| ## | 1900 | -13280.400 | 0.411 | 0.261 |                                  |
| ## | 2000 | -11047.953 | 0.400 | 0.261 |                                  |
| ## | 2100 | -10990.954 | 0.351 | 0.210 |                                  |
| ## | 2200 | -12551.858 | 0.274 | 0.202 |                                  |
| ## | 2300 | -11321.109 | 0.253 | 0.168 |                                  |
| ## | 2400 | -11285.965 | 0.187 | 0.152 |                                  |
| ## | 2500 | -11491.345 | 0.166 | 0.130 |                                  |
| ## | 2600 | -10620.650 | 0.142 | 0.124 |                                  |
| ## | 2700 | -10910.025 | 0.130 | 0.115 |                                  |
| ## | 2800 | -10661.682 | 0.123 | 0.109 |                                  |
| ## | 2900 | -11120.377 | 0.123 | 0.109 |                                  |
| ## | 3000 | -10860.774 | 0.116 | 0.097 |                                  |
| ## | 3100 | -11093.476 | 0.107 | 0.093 |                                  |
| ## | 3200 | -10715.081 | 0.090 | 0.082 |                                  |
| ## | 3300 | -11023.889 | 0.085 | 0.061 |                                  |
| ## | 3400 | -11390.938 | 0.084 | 0.041 |                                  |
| ## | 3500 | -10930.070 | 0.073 | 0.041 |                                  |
| ## | 3600 | -10801.000 | 0.067 | 0.035 |                                  |
| ## | 3700 | -10723.263 | 0.051 | 0.032 |                                  |
| ## | 3800 | -14133.500 | 0.059 | 0.032 |                                  |
| ## | 3900 | -12468.446 | 0.061 | 0.032 |                                  |
| ## | 4000 | -10907.236 | 0.058 | 0.032 |                                  |
| ## | 4100 | -10623.544 | 0.059 | 0.032 |                                  |
| ## | 4200 | -11061.843 | 0.054 | 0.032 |                                  |
| ## | 4300 | -10826.906 | 0.050 | 0.028 |                                  |
| ## | 4400 | -10680.701 | 0.051 | 0.028 |                                  |
| ## | 4500 | -12379.898 | 0.057 | 0.032 |                                  |
| ## | 4600 | -10992.091 | 0.059 | 0.032 |                                  |
| ## | 4700 | -11258.925 | 0.059 | 0.032 |                                  |
| ## | 4800 | -11090.772 | 0.058 | 0.032 |                                  |
| ## | 4900 | -11186.467 | 0.057 | 0.028 |                                  |
| ## | 5000 | -11224.305 | 0.056 | 0.028 |                                  |
| ## | 5100 | -10747.353 | 0.057 | 0.032 |                                  |
| ## | 5200 | -10995.459 | 0.056 | 0.028 |                                  |
| ## | 5300 | -10586.543 | 0.057 | 0.032 |                                  |
| ## | 5400 | -10935.713 | 0.057 | 0.032 |                                  |
| ## | 5500 | -10979.149 | 0.055 | 0.027 |                                  |
| ## | 5600 | -11267.969 | 0.055 | 0.027 |                                  |
| ## | 5700 | -12039.138 | 0.058 | 0.032 |                                  |
| ## | 5800 | -10695.215 | 0.052 | 0.032 |                                  |
| ## | 5900 | -10872.982 | 0.047 | 0.027 |                                  |
| ## | 6000 | -11246.272 | 0.041 | 0.027 |                                  |
| ## | 6100 | -10881.756 | 0.041 | 0.032 |                                  |
| ## | 6200 | -11137.645 | 0.041 | 0.026 |                                  |
| ## | 6300 | -10619.498 | 0.042 | 0.032 |                                  |
| ## | 6400 | -12239.023 | 0.048 | 0.033 |                                  |
| ## | 6500 | -11116.857 | 0.046 | 0.033 |                                  |
| ## | 6600 | -10754.301 | 0.041 | 0.033 |                                  |

|    |       |            |       |       |
|----|-------|------------|-------|-------|
| ## | 6700  | -10607.985 | 0.041 | 0.033 |
| ## | 6800  | -10695.239 | 0.041 | 0.033 |
| ## | 6900  | -10566.238 | 0.041 | 0.033 |
| ## | 7000  | -10645.727 | 0.041 | 0.033 |
| ## | 7100  | -11613.518 | 0.043 | 0.033 |
| ## | 7200  | -10860.047 | 0.045 | 0.033 |
| ## | 7300  | -11704.011 | 0.047 | 0.033 |
| ## | 7400  | -10787.176 | 0.050 | 0.034 |
| ## | 7500  | -11419.568 | 0.052 | 0.049 |
| ## | 7600  | -10641.226 | 0.055 | 0.055 |
| ## | 7700  | -10585.787 | 0.052 | 0.049 |
| ## | 7800  | -11031.619 | 0.047 | 0.040 |
| ## | 7900  | -11226.963 | 0.047 | 0.040 |
| ## | 8000  | -11034.647 | 0.047 | 0.040 |
| ## | 8100  | -10840.425 | 0.046 | 0.040 |
| ## | 8200  | -10595.685 | 0.046 | 0.040 |
| ## | 8300  | -10911.559 | 0.045 | 0.034 |
| ## | 8400  | -10925.692 | 0.038 | 0.029 |
| ## | 8500  | -10598.270 | 0.035 | 0.029 |
| ## | 8600  | -10724.914 | 0.034 | 0.023 |
| ## | 8700  | -10631.230 | 0.033 | 0.023 |
| ## | 8800  | -11099.657 | 0.035 | 0.029 |
| ## | 8900  | -10974.424 | 0.035 | 0.029 |
| ## | 9000  | -11274.696 | 0.036 | 0.029 |
| ## | 9100  | -10644.674 | 0.035 | 0.029 |
| ## | 9200  | -10981.945 | 0.033 | 0.029 |
| ## | 9300  | -11352.458 | 0.031 | 0.029 |
| ## | 9400  | -10711.382 | 0.030 | 0.029 |
| ## | 9500  | -10854.075 | 0.028 | 0.027 |
| ## | 9600  | -10607.410 | 0.025 | 0.023 |
| ## | 9700  | -10916.132 | 0.026 | 0.027 |
| ## | 9800  | -10831.572 | 0.025 | 0.023 |
| ## | 9900  | -11189.264 | 0.025 | 0.027 |
| ## | 10000 | -10895.491 | 0.026 | 0.027 |
| ## | 10100 | -10722.197 | 0.026 | 0.027 |
| ## | 10200 | -11296.157 | 0.027 | 0.028 |
| ## | 10300 | -10844.817 | 0.028 | 0.028 |
| ## | 10400 | -10754.296 | 0.028 | 0.028 |
| ## | 10500 | -11114.064 | 0.028 | 0.028 |
| ## | 10600 | -11366.677 | 0.029 | 0.028 |
| ## | 10700 | -10828.667 | 0.031 | 0.031 |
| ## | 10800 | -10872.635 | 0.029 | 0.028 |
| ## | 10900 | -11257.776 | 0.030 | 0.031 |
| ## | 11000 | -10703.107 | 0.031 | 0.032 |
| ## | 11100 | -10606.090 | 0.029 | 0.031 |
| ## | 11200 | -10735.083 | 0.028 | 0.028 |
| ## | 11300 | -10572.486 | 0.027 | 0.027 |
| ## | 11400 | -10586.426 | 0.024 | 0.023 |
| ## | 11500 | -10884.904 | 0.025 | 0.027 |
| ## | 11600 | -11322.435 | 0.026 | 0.027 |
| ## | 11700 | -10742.602 | 0.027 | 0.027 |
| ## | 11800 | -10723.121 | 0.027 | 0.027 |
| ## | 11900 | -10716.454 | 0.025 | 0.027 |
| ## | 12000 | -10601.800 | 0.024 | 0.022 |

```

##    12100      -10590.421          0.023          0.022
##    12200      -11011.742          0.023          0.022
##    12300      -10982.724          0.021          0.015
##    12400      -10625.147          0.022          0.022
##    12500      -10590.074          0.021          0.015
##    12600      -10708.871          0.020          0.012
##    12700      -10933.288          0.019          0.012
##    12800      -10912.843          0.018          0.012
##    12900      -10617.921          0.018          0.012
##    13000      -10888.835          0.017          0.012
##    13100      -10807.142          0.017          0.012
##    13200      -10971.214          0.017          0.015
##    13300      -10813.319          0.017          0.015
##    13400      -11026.288          0.018          0.015
##    13500      -10695.736          0.018          0.015
##    13600      -10748.607          0.016          0.015
##    13700      -11106.863          0.015          0.015
##    13800      -10591.185          0.017          0.015
##    13900      -10565.620          0.018          0.015
##    14000      -10956.637          0.019          0.019
##    14100      -10847.899          0.019          0.019
##    14200      -10699.148          0.018          0.015
##    14300      -10722.535          0.018          0.015
##    14400      -10652.613          0.017          0.015
##    14500      -10625.497          0.017          0.015
##    14600      -10726.562          0.017          0.015
##    14700      -10568.793          0.016          0.015
##    14800      -10575.345          0.016          0.015
##    14900      -10687.939          0.015          0.014
##    15000      -10678.009          0.014          0.011
##    15100      -10729.511          0.014          0.011
##    15200      -11204.188          0.015          0.011
##    15300      -10847.069          0.016          0.011
##    15400      -10784.117          0.016          0.010
##    15500      -10868.664          0.014          0.009  MEDIAN ELBO CONVERGED
## Drawing a sample of size 4000 from the approximate posterior...
## COMPLETED.
## Finished in 11.9 seconds.

```

```
draws1_vi <- fit1_vi$draws(c("beta0", "beta1", "beta2"), format = "df")
```

```

# Model 2
fit2_vi <- mod2$variational(
  data = stan_data_m2,
  seed = 1,
  algorithm = "fullrank",
  iter = 20000,
  output_samples = 4000
)

```

```

## -----
## EXPERIMENTAL ALGORITHM:
## This procedure has not been thoroughly tested and may be unstable

```



```
## or buggy. The interface is subject to change.
## -----
## Rejecting initial value:
## Log probability evaluates to log(0), i.e. negative infinity.
## Stan can't start sampling from this initial value.
## Rejecting initial value:
## Log probability evaluates to log(0), i.e. negative infinity.
## Stan can't start sampling from this initial value.
## Rejecting initial value:
## Log probability evaluates to log(0), i.e. negative infinity.
## Stan can't start sampling from this initial value.
## Gradient evaluation took 0.000326 seconds
## 1000 transitions using 10 leapfrog steps per transition would take 3.26 seconds.
## Adjust your expectations accordingly!
## Begin eta adaptation.
## Iteration: 1 / 250 [ 0%] (Adaptation)
## Iteration: 50 / 250 [ 20%] (Adaptation)
## Iteration: 100 / 250 [ 40%] (Adaptation)
## Iteration: 150 / 250 [ 60%] (Adaptation)
## Iteration: 200 / 250 [ 80%] (Adaptation)
## Iteration: 250 / 250 [100%] (Adaptation)
## Success! Found best value [eta = 0.1].
## Begin stochastic gradient ascent.
## iter      ELBO    delta_ELBO_mean  delta_ELBO_med  notes
## 100      -5839976.446      1.000      1.000
## 200      -5821754.152      0.502      1.000
## 300      -5853652.391      0.336      0.005  MEDIAN ELBO CONVERGED
## Drawing a sample of size 4000 from the approximate posterior...
## COMPLETED.
## Finished in 8.2 seconds.
```

```
draws2_vi <- fit2_vi$draws(c("mu_alpha", "sigma_alpha", "beta1", "beta2"), format = "df")
```

```
fit1_vi$summary(c("beta0", "beta1", "beta2"))
```

```
## # A tibble: 3 x 7
##   variable    mean  median      sd    mad      q5      q95
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 beta0    -0.960  -0.960  0.00234  0.00241 -0.964  -0.957
## 2 beta1    -0.00643 -0.00644 0.000842 0.000840 -0.00780 -0.00501
## 3 beta2    -0.00205 -0.00205 0.00153  0.00154 -0.00456  0.000450
```

```
fit2_vi$summary(c("mu_alpha", "sigma_alpha", "beta1", "beta2"))
```

```
## # A tibble: 4 x 7
##   variable    mean median      sd    mad      q5      q95
##   <chr>      <dbl>   <dbl>   <dbl>   <dbl>   <dbl>   <dbl>
## 1 mu_alpha    0.310  0.322  0.880  0.875 -1.13    1.76
## 2 sigma_alpha  4.86   3.43  4.71  2.61  0.892  13.4
## 3 beta1     -0.638 -0.612 0.912  0.908 -2.11    0.843
## 4 beta2     -0.934 -0.947 0.934  0.939 -2.46    0.587
```

```
# Age effect
mean(draws1_vi$beta1 > 0)
```

```
## [1] 0
```

```
mean(draws1_vi$beta2 < 0)
```

```
## [1] 0.91375
```

```
mean(draws2_vi$beta1 > 0)
```

```
## [1] 0.23975
```

```
mean(draws2_vi$beta2 < 0)
```

```
## [1] 0.84125
```

```
# peak age
draws1_vi$peak_age <- age_mean + (-draws1_vi$beta1 / (2 * draws1_vi$beta2))
mean(draws1_vi$peak_age)
```

```
## [1] 28.68516
```

```
quantile(draws1_vi$peak_age, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%     97.5%
## 16.18700 28.70752 40.24020
```

```
draws2_vi$peak_age <- age_mean + (-draws2_vi$beta1 / (2 * draws2_vi$beta2))
mean(draws2_vi$peak_age)
```

```
## [1] -196.6115
```

```
quantile(draws2_vi$peak_age, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%     97.5%
## 24.22447 29.88676 34.48903
```

check whether peak age 2 is negative

```
summary(draws2_vi$beta2)
```

```
##      Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
## -4.7135 -1.5656 -0.9471 -0.9336 -0.2970  2.6219
```

```
summary (draws2_vi$peak_age)
```

```
##      Min.   1st Qu.   Median     Mean   3rd Qu.    Max.
## -905012.4    29.5     29.9    -196.6    30.2     456.7
```

```
mean(draws2_vi$beta2 > 0)
```

```
## [1] 0.15875
```

```
mean(abs(draws2_vi$beta2) < 0.05)
```

```
## [1] 0.031
```

```
sum(draws2_vi$peak_age < 0)
```

```
## [1] 19
```

```
sum(draws2_vi$peak_age > 100)
```

```
## [1] 3
```

```
# hierarchical heterogeneity
mean(draws2_vi$sigma_alpha)
```

```
## [1] 4.855929
```

```
quantile(draws2_vi$sigma_alpha, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%      97.5%
## 0.6977758 3.4345237 17.0602015
```

Computation by Laplace

```
library(rstan)
```

```
## Warning: package 'rstan' was built under R version 4.3.3
```

```
## Loading required package: StanHeaders
```

```
## Warning: package 'StanHeaders' was built under R version 4.3.3
```

```
##
## rstan version 2.32.7 (Stan version 2.32.2)
```

```
## For execution on a local, multicore CPU with excess RAM we recommend calling
## options(mc.cores = parallel::detectCores()).
## To avoid recompilation of unchanged Stan programs, we recommend calling
## rstan_options(auto_write = TRUE)
## For within-chain threading using 'reduce_sum()' or 'map_rect()' Stan functions,
## change 'threads_per_chain' option:
## rstan_options(threads_per_chain = 1)
```

```
##
## Attaching package: 'rstan'
```

```
## The following objects are masked from 'package:posterior':
##
##     ess_bulk, ess_tail
```

```
stan_mod1 <- stan_model("model1.stan")
stan_mod2 <- stan_model("model2.stan")
```

```
lap1 <- optimizing(
  object = stan_mod1,
  data = stan_data_m1,
  seed = 1,
  hessian = TRUE,
  as_vector = FALSE
)
```

```
## Warning in .local(object, ...): non-zero return code in optimizing
```

```
lap2 <- optimizing(
  object = stan_mod2,
  data = stan_data_m2,
  seed = 1,
  hessian = TRUE,
  as_vector = FALSE
)
```

```
## Warning in .local(object, ...): non-zero return code in optimizing
```

```
Model1
```

```
lap1$par[c("beta0", "beta1", "beta2")]
```

```
## $beta0
## [1] 0.6598697
##
## $beta1
## [1] 0.1945492
##
## $beta2
## [1] -1.624643
```

```
dim(lap1$hessian)
```

```
## [1] 3 3
```

```
lap1$hessian[1:3, 1:3]
```

```
##          beta0          beta1          beta2
## beta0 -54032.787 -2315.8057 -35172.2707
## beta1 -2315.806 -35172.3091 -713.3609
## beta2 -35172.271 -713.3609 -42469.2739
```

```
cov_m1 <- solve(-lap1$hessian)
cov_m1
```

```
##          beta0          beta1          beta2
## beta0  4.029150e-05 -1.976756e-06 -3.333548e-05
## beta1 -1.976756e-06  2.853813e-05  1.157755e-06
## beta2 -3.333548e-05  1.157755e-06  5.113482e-05
```

```
eigen(cov_m1)$values
```

```
## [1] 7.957829e-05 2.848096e-05 1.190519e-05
```

```
library(MASS)
```

```
##
## Attaching package: 'MASS'
```

```
## The following object is masked from 'package:dplyr':
##
##      select
```

```
mode_m1 <- unlist(lap1$par[c("beta0", "beta1", "beta2")])
```

```
set.seed(1)
```

```
draws1_lap <- as.data.frame(
  MASS::mvrnorm(
    n = 4000,
    mu = mode_m1,
    Sigma = cov_m1
  )
)
```

```
colnames(draws1_lap) <- c("beta0", "beta1", "beta2")
```

```
mean(draws1_lap$beta1 > 0)
```

```
## [1] 1
```

```
mean(draws1_lap$beta2 < 0)
```

```
## [1] 1
```

```
draws1_lap$peak_age <- age_mean + (-draws1_lap$beta1 / (2 * draws1_lap$beta2))
```

```
mean(draws1_lap$peak_age)
```

```
## [1] 30.14915
```

```
quantile(draws1_lap$peak_age, c(0.025, 0.5, 0.975))
```

```
##      2.5%      50%      97.5%  
## 30.14589 30.14915 30.15244
```

Model2

```
lap2$par[c("mu_alpha", "sigma_alpha", "beta1", "beta2")]
```

```
## $mu_alpha  
## [1] -1.319163  
##  
## $sigma_alpha  
## [1] 1.834913  
##  
## $beta1  
## [1] 0.1625231  
##  
## $beta2  
## [1] -1.679341
```

```
dim(lap2$hessian)
```

```
## [1] 204 204
```

```
lap2$hessian[1:8, 1:8]
```

```
##           mu_alpha  sigma_alpha  alpha.1  alpha.2  alpha.3  
## mu_alpha -59.4417227 -148.1887789  0.2970086  0.2970086  0.2970086  
## sigma_alpha -148.1887789 -344.5403029 -0.3399007  0.4856726  0.6360645  
## alpha.1      0.2970086 -0.3399007 -83.8550978  0.0000000  0.0000000  
## alpha.2      0.2970086  0.4856726  0.0000000 -176.0865441  0.0000000  
## alpha.3      0.2970086  0.6360645  0.0000000  0.0000000 -239.0673442  
## alpha.4      0.2970086  0.1572570  0.0000000  0.0000000  0.0000000  
## alpha.5      0.2970086  0.3290322  0.0000000  0.0000000  0.0000000  
## alpha.6      0.2970086 -0.2992136  0.0000000  0.0000000  0.0000000  
##           alpha.4  alpha.5  alpha.6  
## mu_alpha      0.2970086  0.2970086  0.2970086  
## sigma_alpha    0.1572570  0.3290322 -0.2992136
```

```
## alpha.1      0.0000000    0.0000000    0.0000000
## alpha.2      0.0000000    0.0000000    0.0000000
## alpha.3      0.0000000    0.0000000    0.0000000
## alpha.4     -176.9884601    0.0000000    0.0000000
## alpha.5       0.0000000 -200.9851498    0.0000000
## alpha.6       0.0000000    0.0000000 -72.6713258
```

```
cov_m2 <- solve(-lap2$hessian)
eigen(cov_m2)$values
```

```
## [1] 3.473331e-02 2.993818e-02 2.927712e-02 2.338631e-02 2.093879e-02
## [6] 1.886372e-02 1.798832e-02 1.722465e-02 1.716795e-02 1.710309e-02
## [11] 1.634280e-02 1.578634e-02 1.571216e-02 1.567917e-02 1.539725e-02
## [16] 1.438836e-02 1.376989e-02 1.340185e-02 1.316137e-02 1.307168e-02
## [21] 1.275031e-02 1.262731e-02 1.230852e-02 1.218717e-02 1.192731e-02
## [26] 1.181589e-02 1.176543e-02 1.169506e-02 1.164961e-02 1.147588e-02
## [31] 1.098792e-02 1.083689e-02 1.067258e-02 1.057863e-02 1.053293e-02
## [36] 1.045755e-02 1.037494e-02 1.016307e-02 1.011752e-02 9.957141e-03
## [41] 9.901968e-03 9.869706e-03 9.562339e-03 9.463510e-03 9.434155e-03
## [46] 9.423513e-03 9.355014e-03 9.293417e-03 9.079044e-03 8.897072e-03
## [51] 8.819039e-03 8.691448e-03 8.579549e-03 8.491243e-03 8.329934e-03
## [56] 8.177882e-03 8.126992e-03 8.015325e-03 7.934703e-03 7.677579e-03
## [61] 7.608464e-03 7.560796e-03 7.458064e-03 7.349622e-03 7.325194e-03
## [66] 7.274135e-03 7.025419e-03 6.965071e-03 6.870304e-03 6.786219e-03
## [71] 6.651655e-03 6.578469e-03 6.501062e-03 6.486560e-03 6.454391e-03
## [76] 6.427399e-03 6.386295e-03 6.324629e-03 6.248775e-03 6.226601e-03
## [81] 6.188353e-03 6.149616e-03 6.107649e-03 6.079874e-03 6.047779e-03
## [86] 6.031131e-03 5.964444e-03 5.910339e-03 5.854097e-03 5.734320e-03
## [91] 5.677042e-03 5.643085e-03 5.584781e-03 5.564900e-03 5.556855e-03
## [96] 5.520961e-03 5.481448e-03 5.372539e-03 5.283377e-03 5.254102e-03
## [101] 5.223711e-03 5.201418e-03 5.163060e-03 5.110784e-03 5.084543e-03
## [106] 5.035883e-03 4.988961e-03 4.963008e-03 4.931032e-03 4.903724e-03
## [111] 4.835018e-03 4.760443e-03 4.709253e-03 4.650595e-03 4.610436e-03
## [116] 4.555652e-03 4.526233e-03 4.481089e-03 4.449540e-03 4.396982e-03
## [121] 4.342616e-03 4.332276e-03 4.308415e-03 4.216914e-03 4.203374e-03
## [126] 4.182062e-03 4.164444e-03 4.150679e-03 4.132076e-03 4.117917e-03
## [131] 4.042157e-03 4.032699e-03 4.016341e-03 3.984345e-03 3.968052e-03
## [136] 3.944235e-03 3.925859e-03 3.909982e-03 3.898337e-03 3.885499e-03
## [141] 3.858531e-03 3.852568e-03 3.821123e-03 3.807792e-03 3.796342e-03
## [146] 3.772819e-03 3.706567e-03 3.670115e-03 3.645845e-03 3.638317e-03
## [151] 3.623965e-03 3.597533e-03 3.559513e-03 3.540631e-03 3.523815e-03
## [156] 3.509097e-03 3.497823e-03 3.480740e-03 3.447487e-03 3.425402e-03
## [161] 3.399201e-03 3.389341e-03 3.371184e-03 3.358640e-03 3.337569e-03
## [166] 3.323233e-03 3.304366e-03 3.284228e-03 3.268503e-03 3.265702e-03
## [171] 3.231692e-03 3.213803e-03 3.202223e-03 3.129615e-03 3.100792e-03
## [176] 3.052538e-03 3.017525e-03 3.011260e-03 2.996749e-03 2.968416e-03
## [181] 2.938826e-03 2.911647e-03 2.887841e-03 2.869189e-03 2.859683e-03
## [186] 2.849111e-03 2.831932e-03 2.815913e-03 2.807552e-03 2.799438e-03
## [191] 2.795626e-03 2.736849e-03 2.715981e-03 2.702509e-03 2.695583e-03
## [196] 2.688666e-03 2.666115e-03 2.598823e-03 2.578536e-03 2.448605e-03
## [201] 2.313910e-03 4.120315e-05 3.362995e-05 -2.673669e-01
```

```
keep <- c("mu_alpha", "sigma_alpha", "beta1", "beta2")
cov_m2_sub <- cov_m2[keep, keep]
eigen(cov_m2_sub)$values
```

```
## [1] 2.464928e-03 7.695687e-05 4.236570e-05 -2.670571e-01
```

```
mode_mu_alpha <- unlist(lap2$par["mu_alpha"])
mode_sigma_alpha <- unlist(lap2$par["sigma_alpha"])
mode_beta1 <- unlist(lap2$par["beta1"])
mode_beta2 <- unlist(lap2$par["beta2"])
mode_alpha <- unlist(lap2$par["alpha"])
```

```
mode_m2 <- c(
  mu_alpha = mode_mu_alpha,
  sigma_alpha = mode_sigma_alpha,
  alpha = mode_alpha,
  beta1 = mode_beta1,
  beta2 = mode_beta2
)
```

```
length(mode_m2)
```

```
## [1] 204
```

```
#set.seed(1)
```

```
#draws2_lap <- as.data.frame(
  #MASS::mvrnorm(
    #n = 4000,
    #mu = mode_m2,
    # Sigma = cov_m2
  # )
#)
```

```
#mean(draws2_lap$beta1 > 0)
#mean(draws2_lap$beta2 < 0)
```

```
#draws2_lap$peak_age <- age_mean + (-draws2_lap$beta1 / (2 * draws2_lap$beta2))
```

```
#mean(draws2_lap$peak_age)
#quantile(draws2_lap$peak_age, c(0.025, 0.5, 0.975))
```

The fact that the covariance matrix corresponding to the Hessian of the eigenvalues is negative indicates that the local Gaussian approximation itself is unstable for this hierarchical posterior.

““